

Functional Specification for Decentralized DNS project

Author	Mohammed Awawdi, Ibrahim Kamel, Ahmad Ghanayim
Version	V0.1
Created	13.04.2026
Last Updated	13.04.2026

1 Overview

This project involves the design and implementation of a brand-new decentralized Domain Name System (DNS) and Public Key Infrastructure (PKI) built on a blockchain network. The system replaces central authorities with a decentralized application (dApp) that handles namespace registration and routing. Furthermore, it integrates a BitTorrent peer-to-peer network to store and serve static resources natively "inside" the DNS, maintaining strict data integrity through cryptographic hashes stored directly on the blockchain ledger.

1.1 Detailed Definitions

- **dApp (Decentralized Application):** The core smart contract logic residing on the blockchain that manages namespace ownership, updates, and the collection of fees.
- **Resolver Server:** A custom-built server acting as the intermediary between clients and the decentralized infrastructure. It answers client queries, caches data, interacts with the blockchain, and participates in the P2P network.
- **Resource Reference:** A specialized DNS record type that stores a cryptographic SHA hash pointing to a large static file hosted on the BitTorrent network, bypassing blockchain storage limits.
- **Zero-Knowledge Proofs (ZKP) / Cryptographic Signatures:** Cryptographic mechanisms used to prove the authenticity and correctness of DNS query responses, acting as a native, secure alternative to DNSSEC and preventing Sybil attacks.

1.2 Operating platforms

- **Blockchain Environment:** A local blockchain network (e.g., Hardhat or Truffle) for active development, transitioning to a public Testnet for final deployment and demonstration.
- **Resolver Environment:** A standard Linux/Unix environment running a backend runtime (such as Node.js or Python).
- **P2P Network:** The standard BitTorrent protocol, utilizing existing client architectures.

1.3 Interfaces

- **Resolver API:** A RESTful API to process incoming client queries, manage cache data, and accept namespace update requests.
- **Blockchain Interface:** Web3 RPC APIs (e.g., ethers.js, web3.js) for communication between the Resolver Server and the Solidity smart contracts.
- **P2P Interface:** BitTorrent protocol communication for seeding and retrieving static files.

2 Features

2.1 Main features

- **Decentralized Namespace Management:** Registration, purchasing, and updating of domains on the blockchain via native cryptocurrency fees.
- **Integrated PKI Security:** Built-in cryptographic proofs ensuring the absolute truthfulness of DNS responses.
- **Extended Query Capabilities:** Support for standard DNS routing alongside advanced queries dictating specific ports, transport layer protocols (UDP/TCP/QUIC), and service types (HTTP/SMTP).
- **Dynamic Record Expansion:** The ability to dynamically add new query types and define mandatory/optional fields.
- **P2P Static Resource Storage:** Full integration of the "Resource Reference" field, bridging the blockchain hash registry with the BitTorrent file-sharing network.
- **Caching Resolver Server:** A functional node that handles TTL-based caching, acts as a gateway for blockchain updates, and serves large file data to the end client.

2.2 Secondary features

- **UDP Protocol Integration:** Extending the Resolver Server to accept lightweight, high-speed queries over UDP alongside the RESTful API.
- **Resource Type Validation:** A mechanism ensuring that files uploaded to the P2P network (like HTML or JS) match their registered type, potentially utilizing trusted verification endpoints.

2.3 Nice-to-have features

- **Resolver Incentive Mechanism:** A system rewarding resolver servers financially based on their query volume.
Reason not implemented: Designing an exploit-free economic model requires substantial time and extends beyond the core networking requirements.
- **Decentralized Web Browsing Integration:** Configuring a standard web browser to natively resolve and render decentralized static websites through the custom infrastructure.
Reason not implemented: Browser-level HTTP interception and extension development divert focus from the primary blockchain and protocol goals.

3 Dependencies

- **Solidity Frameworks:** Tools for writing, compiling, and deploying the dApp.
- **Web3 Integration Libraries:** For backend-to-blockchain communication.
- **BitTorrent Libraries:** Existing clients/APIs to manage P2P file transfers programmatically.
- **Cryptographic Libraries:** Dependencies for generating and verifying zero-knowledge proofs.

4 Assumptions

- The system operates independently as a new standard; it does not need to be backward compatible with or operate in parallel to the global legacy DNS infrastructure.
- Reliable, open-source BitTorrent clients are available and suitable for integration into the Resolver Server.

5 Related Skills Evaluation

An assessment of the skills required for executing the project. This allows you to evaluate what things you *don't* know; alongside the things you *do* know (and to what extent).

5.1 Previous Experience

- **Internet Communication:** Strong foundational knowledge of standard DNS, HTTP, TCP/UDP protocols, and caching mechanisms.
- **Basic Cryptography:** Familiarity with SHA hashing and public key infrastructure concepts.
- **BitTorrent Internals:** Understanding Distributed Hash Tables (DHT) and peer discovery within P2P networks.

5.2 Learning “Theory”

- **Blockchain Architecture:** Understanding decentralized ledgers, gas fees, state modifications, and defense mechanisms against Sybil attacks.
- **Zero-Knowledge Proofs:** The mathematics and practical application of verifying data integrity without exposing underlying state variables.

5.3 Learning Frameworks

- **Solidity Development:** Syntax, security paradigms, and smart contract design patterns.
- **Web3 Backend Integration:** Bridging traditional server environments with decentralized networks.
- **P2P Implementation:** Integrating BitTorrent protocols programmatically into a custom backend service.